



UNIVERSIDAD REY JUAN CARLOS

MÁSTER UNIVERSITARIO EN ANÁLISIS DE DATOS DEPORTIVOS

Práctica Final

Sistema para el análisis en tiempo real de eventos en partidos de fútbol

Daniel López Paredes

Fecha: 12/05/2026

Asignatura: Big Data Processing II

Profesor: Alberto Fernández Isabel, Natalia Madrueño Sierro, Rubén Rodríguez Fernández

ÍNDICE

1.	Introducción	1
2.	Objetivos	1
3.	Arquitectura general.....	1
4.	Descripción de los datos.....	2
5.	Uso de Kafka.....	2
6.	Procesamiento con Spark	3
7.	Métricas calculadas	4
8.	Diseño del módulo RAG.....	6
9.	Diseño del flujo en LangGraph	6
10.	Tools implementadas y función de cada una	7
11.	Visualización y Generación del Informe.....	8
12.	Decisiones de optimización.....	12
13.	Limitaciones del sistema	13
14.	Conclusiones	13
15.	Consideraciones finales.....	14

1. Introducción

El análisis de datos en tiempo real es fundamental para poder tener información procesada en el menor tiempo posible desde que se generó. En el ámbito deportivo tiene un papel muy importante ya que disponer de información antes que los rivales pueden generar una ventaja competitiva muy grande y decidir el resultado final de una competición.

Por ello en este trabajo se diseña y desarrolla la arquitectura para un sistema que obtenga, procese y muestre datos de eventos de partidos de fútbol en tiempo real. A través de tecnologías y lenguajes de programación como Python, Kafka, Spark Streaming, Streamlit o el uso de la IA Generativa (GenAI) y agentes se pretende construir un sistema completo capaz de extraer información en base a los datos de eventos y mostrarla en tiempo real.

En los siguientes apartados se explicará en detalle el uso y aplicación de cada una de estas tecnologías y que impacto tienen en el desarrollo del proyecto. Se analizará todo el flujo de trabajo desde la obtención de los datos hasta su visualización en una interfaz de usuario de forma dinámica.

2. Objetivos

El objetivo del proyecto es construir un sistema end-to-end de analítica deportiva en tiempo real. Partiendo de datos estáticos de eventos, el sistema pretende simular que se están produciendo en tiempo real.

A través de Kafka y Spark Streaming se busca obtener y procesar estos datos para calcular métricas y agregaciones durante la generación de los diferentes eventos. Estas métricas y eventos procesados se persisten para permitir a otras aplicaciones realizar informes o visualizarlos en alguna interfaz de usuario

Además, se busca potenciar el análisis mediante el desarrollo de un módulo GenAI. Este módulo permitirá generar resúmenes mediante las métricas ya preparadas y procesadas, recuperar contexto documental mediante RAG y genera fragmentos narrativos cada vez que se cierra una ventana de partido (establecida por defecto cada 5 minutos).

3. Arquitectura general

La arquitectura separa ingesta, procesamiento, persistencia, generación textual y visualización. A modo de resumen, hay un productor que publica datos en un bróker Kafka; Spark consume los datos y concentra el procesamiento; Parquet actúa como formato de persistencia; el grafo de agentes produce fragmentos narrativos; y Streamlit presenta los artefactos ya materializados.

En detalle estos son los elementos de la arquitectura y su función en el sistema:

- **Kafka:** broker en modo KRaft usado como bus real de ingesta en tiempo real.
- **Kafka Producer:** normaliza eventos StatsBomb y los publica por lotes en el topic *match_events*.
- **Spark Streaming:** consume Kafka, valida eventos, calcula métricas y escribe snapshots Parquet.
- **LangGraph:** coordina el único grafo actual: *metrics_agent* -> *rag_agent* -> *writer_agent*.

- **RAG:** recupera contexto documental desde `data/docs/rag_corpus` mediante Chroma o TF-IDF.
- **Streamlit:** lee Parquet/PDF ya generados y muestra métricas, fragmentos, trazas y descarga.

4. Descripción de los datos

La fuente principal es un archivo en formato JSON en la ruta `data/static/events.json`, del proveedor de datos de StatsBomb el cual contiene todos los eventos de un partido del Real Madrid contra el Espanyol de la temporada 2024/2025. Para cada evento se tiene información como quien realizó la acción, las coordenadas x, y del evento o el resultado del mismo (un pase fue exitoso, un tiro fue a puerta, gol o fuera, etc.).

Por ello, antes de enviar los eventos al bróker de Kafka, el productor transforma cada evento relevante en un esquema plano con `event_id`, `timestamp`, `match_id`, `team`, `player`, `event_type`, `zone`, `minute`, `second`, `outcome`, `value` y `period`. La zona se deriva de coordenadas de campo y `outcome` se normaliza para facilitar el análisis.

Además, se cuenta con otro archivo JSON con información estática sobre el partido (`data/static/match.json`) con datos sobre la fecha, hora y localización del partido, asistencia o el árbitro. Y un último archivo JSON con información sobre los equipos (`data/static/teams.json`). Estos archivos solo serán utilizados por el visualizador para dar más información y contexto sobre el partido y los equipos a la simulación de eventos en tiempo real.

Por otro lado, en la carpeta `rag_corpus` se encuentran archivos que ayudaran al LLM a como interpretar los resultados del análisis. Por último en la carpeta `output` se persistirán todos los datos procesados y cálculos de métricas que procese Spark Streaming de los eventos publicados en el bróker de Kafka

5. Uso de Kafka

Kafka se emplea como mecanismo real de ingesta de datos y actúa como capa de comunicación entre la producción simulada de eventos en tiempo real y el procesamiento posterior mediante Spark Structured Streaming. En este proyecto, Kafka permite desacoplar el componente que genera los eventos del partido del componente que los consume y transforma, reproduciendo una arquitectura habitual en sistemas de analítica en streaming.

El flujo parte de eventos de partido en formato StatsBomb almacenados en `data/static/events.json`. Estos eventos no se procesan directamente por Spark, sino que primero son leídos, normalizados y publicados progresivamente en Kafka por el productor. Posteriormente, el pipeline de streaming consume dichos mensajes desde el topic configurado y los transforma en artefactos analíticos persistidos en Parquet. Es importante distinguir tres elementos principales dentro del uso de Kafka en el proyecto:

- **Broker Kafka:** es el servicio central encargado de recibir, almacenar temporalmente y distribuir los eventos publicados. En este proyecto se utiliza como bus de eventos entre el productor y el consumidor, permitiendo que ambos componentes funcionen de forma desacoplada. El topic principal utilizado es: **match_events**. Este topic representa el canal por el que circulan los eventos del partido. El productor escribe mensajes en él y el pipeline de streaming los lee para su procesamiento. Gracias a Kafka, el sistema no depende de que el productor y el consumidor estén perfectamente sincronizados. El

productor puede publicar eventos y Spark puede consumirlos desde el inicio del topic usando la política **startingOffsets=earliest**, lo que permite recuperar eventos ya publicados si el consumidor arranca después.

- **Publicador Kafka (*kafka_producer.py*):** es el componente responsable de simular la llegada de eventos en tiempo real. Su entrada principal es el fichero estático de eventos StatsBomb: *data/static/events.json*. Aunque los datos originales están almacenados en un JSON, el productor no los publica directamente en bruto. Antes de enviarlos a Kafka, realiza una fase de preprocesamiento cuyo objetivo es convertir los eventos a un esquema más simple, estable y adecuado para streaming. Entre las acciones principales que realiza se encuentran:
 - Leer y ordenar los eventos por el campo *index*, conservando la secuencia cronológica del partido.
 - Filtrar únicamente los eventos relevantes para el análisis deportivo, como pases, tiros, presiones, duelos, conducciones, regates, intercepciones, bloqueos y despejes.
 - Aplanar la estructura original de StatsBomb a un esquema común.
 - Calcular la zona del campo a partir de las coordenadas del evento y mapear resultados de las acciones en función de su tipo.
 - Publicar los eventos en Kafka por lotes (en el topic mencionado *match_events*), introduciendo pausas para simular una fuente real-time.

Además, el esquema publicado por el productor constituye el contrato de datos entre Kafka y Spark con los siguientes campos: *event_id*, *timestamp*, *match_id*, *team*, *player*, *event_type*, *zone*, *minute*, *second*, *outcome*, *value*, *period*

- **Consumidor Kafka (*streaming_pipeline.py*):** componente utiliza Spark Structured Streaming para conectarse al topic *match_events*, leer los mensajes publicados y procesarlos en micro-batches. En el siguiente apartado se entra en detalle sobre la implementación de este componente

6. Procesamiento con Spark

El procesamiento reside en **src/streaming_pipeline.py**. Este módulo actúa como consumidor Kafka y como elemento principal para el procesamiento de los datos. Su función es leer los eventos publicados en el topic *match_events*, validarlos, mantener una visión acumulada del partido y generar periódicamente archivos persistentes en formato Parquet. Spark Structured Streaming permanece escuchando nuevos mensajes de Kafka y los procesa en microbatches. Cada microbatch contiene un conjunto reducido de eventos recibidos desde la última iteración del stream, lo que permite simular un procesamiento cercano al tiempo real sin perder la capacidad de aplicar lógica batch sobre cada grupo de datos.

Desde que se consumen los eventos hasta que son procesados y persistidos se llevan a cabo diferentes pasos:

1. **Lectura desde Kafka:** Spark se conecta al broker Kafka y se suscribe al topic principal del proyecto: *match_events*. Además, se elabora un StructType para evitar depender de inferencia automática de tipos y garantiza que todos los eventos sigan el contrato esperado por el sistema.
2. **Validación de registros:** Una vez parseado el JSON, el pipeline descarta aquellos registros que no contienen información mínima para ser procesados correctamente. En concreto, se eliminan eventos sin identificador y eventos que no tienen información útil de equipo o jugador. Esta fase permite que los artefactos posteriores se construyan únicamente a partir de

- 3. Procesamiento por microbatches:** pipeline utiliza *foreachBatch*, una funcionalidad de Spark Structured Streaming que permite aplicar una función personalizada a cada microbatch recibido. En lugar de escribir directamente cada lote en disco, el sistema procesa el microbatch, lo integra con el estado acumulado y decide si corresponde generar un nuevo snapshot.
- 4. Almacén en memoria y deduplicación:** el pipeline mantiene un almacén en memoria indexado por `event_id`. En la implementación, este almacén actúa como una representación acumulada de todos los eventos válidos observados hasta el momento hasta que son persistidos.
- 5. Generación periódica de snapshots:** El pipeline no escribe en disco después de cada microbatch. En su lugar, controla la frecuencia de persistencia mediante una variable (establecida a 30 segundos por defecto). Con esto, cada vez que se procesa un microbatch, el sistema comprueba cuánto tiempo ha pasado desde el último snapshot. Solo si se supera el intervalo configurado se genera una nueva vista consolidada del partido. Este mecanismo evita escrituras excesivas en disco y reduce la carga sobre el sistema de ficheros. Al mismo tiempo, permite que Streamlit y el módulo de reporting tengan datos suficientemente recientes para actualizar la visualización y los informes. En cada snapshot se parte del almacén acumulado en memoria y se construyen tres salidas principales:
`output/processed/events.parquet;` `output/aggregates/team_metrics.parquet;`
`output/report_windows/window_metrics.parquet`
- 6. Cálculo de métricas y generación de informe incremental:** una vez persistidos los eventos limpios en `events.parquet`, el pipeline genera una serie de archivos derivados que alimentan el resto del sistema. En primer lugar, calcula las métricas acumuladas por equipo, agregando todos los eventos disponibles hasta el snapshot actual y se guarda en `team_metrics.parquet`. Para cada ventana del partido, por ejemplo 0-5 o 5-10, el sistema recopila las métricas principales y los eventos más relevantes de ese tramo. Esas métricas pasan por un workflow de agentes que organiza la información y prepara el prompt final para el LLM. Finalmente, el modelo genera un texto breve en lenguaje natural basado únicamente en las métricas y eventos de la ventana, convirtiendo los datos numéricos en una narración interpretable del partido. Por último, los resultados generados se persisten como artefactos finales del sistema. Los segmentos narrativos se guardan en `incremental_segments.parquet`, se actualiza el contexto en `incremental_rag.parquet` y se refresca el informe `match_report.pdf`. Estos ficheros quedan disponibles para que Streamlit los lea y los muestre en el dashboard sin necesidad de acceder directamente a Kafka ni al proceso de Spark.

7. Métricas calculadas

4 | P á g i n a

Las métricas calculadas incluyen indicadores básicos de actividad y también métricas de rendimiento ofensivo y defensivo. Cada una de ellas nos da información sobre diferentes aspectos del juego. Algunas de estas métricas se pueden observar en la siguiente tabla

Familia	Campos	Interpretacion
Dominio	passes, successful_passes, pass_success_pct	Volumen y precision de circulacion.
Amenaza	shots, goals, avg_value	Produccion ofensiva e impacto.
Disputa	duels, successful_duels, duel_success_pct	Control de acciones individuales.
Defensa	pressures, interceptions, blocks, clearances	Carga defensiva y capacidad de contencion.
Indices	offensive_index, defensive_index	Resumen ponderado de ataque y defensa.

En cuanto a los índices actúan como resumen del rendimiento en las fases de ataque y defensa del equipo y se pondera de la siguiente manera:

- $\text{indice_ofensivo} = 0.8 * \text{pases_exitosos} + 0.6 * \text{conducciones} + 1.2 * \text{regates_exitosos} + 2.0 * \text{tiros} + 5.0 * \text{goles}$
- $\text{indice_defensivo} = 0.5 * \text{presiones} + 1.2 * \text{duelos_ganados} + 1.5 * \text{intercepciones_exitosas} + 1.0 * \text{bloqueos} + 0.8 * \text{despejes}$

Estas métricas se generan de 2 formas distintas:

- **Métricas acumuladas por equipo:** representan el estado global del partido hasta el snapshot actual. Se calculan agrupando todos los eventos por equipo y se almacenan en: *output/aggregates/team_metrics.parquet*. Cada snapshot añade una nueva fotografía temporal del rendimiento acumulado de ambos equipos.
- **Métricas por ventanas de tiempo:** el partido se divide en ventanas de 5 minutos por defecto. Las métricas se calculan únicamente con los eventos pertenecientes a cada tramo y se almacenan en: *output/report_windows/window_metrics.parquet*

Todas las métricas se guardan en formato Parquet, ya que es un formato columnar eficiente y fácilmente consumible por Spark, pandas y Streamlit. Estos archivos actúan como capa intermedia de persistencia para que tanto el dashboard como el sistema de generación de reportes puedan trabajar sobre datos ya consolidados, sin necesidad de acceder directamente a Kafka.

8. Diseño del módulo RAG

El módulo RAG del proyecto sirve para enriquecer los textos generados por el LLM con contexto futbolístico documentado. Su objetivo no es calcular métricas ni decidir qué ocurrió en el partido, sino ayudar al agente redactor a interpretar mejor los datos ya procesados por Spark.

El corpus RAG está en: `data/docs/rag_corpus/`. Esto, incluye documentos Markdown con criterios sobre indicadores de rendimiento, guías de redacción y perfiles tácticos. Ahí se explica, por ejemplo, cómo interpretar un alto volumen de pases, un buen porcentaje de acierto, un índice defensivo elevado, duelos ganados, presión o tramos de alta intensidad.

El diseño está implementado principalmente en: `src/rag_pipeline.py`; `src/tools.py`; `src/graph_workflow.py`

El flujo que envuelva a la implementación y uso de RAG en el sistema es:

metrics_agent → rag_agent → writer_agent

Primero, `metrics_agent` analiza las métricas de una ventana cerrada y construye una consulta RAG con los hechos principales: eventos, tiros, goles, presiones, equipo más activo e intensidad.

Después, `rag_agent` usa la tool `retrieve_document_context(query)` para llamar al módulo RAG y recuperar fragmentos relevantes del corpus documental. El contexto recuperado se devuelve con sus fuentes, para que el texto final sea trazable.

Por último, `writer_agent` recibe las métricas, eventos representativos, métricas acumuladas y contexto RAG. Con todo eso construye el prompt final para Ollama y genera un fragmento narrativo.

Para que RAG recupere el contexto, tiene dos modos:

- **Chroma + embeddings:** si están disponibles las dependencias de LangChain, usa Chroma como base vectorial persistente en `output/chroma_db` con el modelo de embeddings `sentence-transformers/all-mpnet-base-v2`
- Si esas dependencias no están disponibles, usa un **fallback local TF-IDF**. Esto permite que el sistema siga funcionando, aunque no esté disponible la parte vectorial.

El RAG aporta conocimiento interpretativo, no datos nuevos. Por ejemplo, si Spark detecta muchos pases exitosos, el RAG ayuda a redactar que eso puede indicar dominio con balón. Si detecta un índice defensivo alto, ayuda a matizar si puede significar buena defensa o simplemente que el equipo está defendiendo mucho. Así, el informe final combina: datos reales del partido, contexto táctico recuperado y redacción del LLM

9. Diseño del flujo en LangGraph

El flujo GenAI actual está diseñado como un grafo simple y trazable. Su objetivo es generar un fragmento narrativo por cada ventana cerrada del partido, por defecto, cada 5 minutos. Esto significa que el sistema trabaja de forma incremental, es decir, cada vez que Spark detecta una ventana cerrada (cada 5 minutos de eventos del partido), se ejecuta este workflow para producir un texto corto de esa ventana.

En cuanto al workflow, el grafo está definido en `src/graph_workflow.py` y la estructura es como la mencionada en el apartado anterior:

metrics_agent → rag_agent → writer_agent → END

Por otro lado, LangGraph utiliza un **estado** llamado `WindowReportState`. Ese estado va acumulando la información que cada agente produce o necesita. Contiene campos como `match_id`, `window_start_minute`, `window_end_minute`, `window_metrics`, `cumulative_metrics`... los cuales tienen información sobre la ventana que se quiere obtener el texto. Esto permite que cada nodo añada información sin perder trazabilidad. Por ejemplo, el primer agente añade la consulta RAG, el segundo añade el contexto recuperado y el tercero añade el texto final.

A continuación, se explica cada nodo en detalle:

- **Metrics Agent (metrics_agent_node):** su función es preparar la ventana con todas las métricas para establecer el contexto en el RAG. El nodo recibe información sobre las métricas de la ventana, las acumuladas hasta el momento y los eventos de la ventana. A partir de las métricas de la ventana identifica hechos relevantes, como: total de eventos, tiros, goles, intensidad del tramo, presiones, equipo más activo defensivamente, equipo con más presión, etc. Con eso construye una consulta textual para el RAG. Por ejemplo:

“interpretación futbolística de ventana de partido: 35 eventos, 2 tiros, 0 goles, 8 presiones, equipo más activo Real Madrid, equipo con más presión Espanyol, intensidad, posesión, presión, rendimiento y redacción para stakeholder”

Este agente no genera texto final. Solo transforma métricas estructuradas en una consulta semántica útil.

- **Rag Agent (rag_agent_node):** su función es recuperar contexto documental. Toma la `rag_query` creada por el agente anterior y llama a la tool: `retrieve_document_context(query)`. Esa tool consulta los documentos de: `data/docs/rag_corpus/` y devuelve fragmentos relevantes con sus fuentes. Este contexto ayuda al agente redactor a interpretar correctamente las métricas. Por ejemplo, le da criterios para saber que muchos pases con alto acierto pueden indicar dominio, o que un índice defensivo alto puede significar buena defensa o mucha exigencia rival.
- **Writer Agent (writer_agent_node):** este último nodo se encarga de construir el prompt final el cual incluye métricas de la ventana, eventos representativos, métricas acumuladas, contexto RAG e instrucciones de redacción. Después llama al LLM basado en un modelo llama:3.3 local mediante Ollama. Si Ollama falla, responde vacío o devuelve datos estructurados, el sistema genera un texto fallback determinista. Así el pipeline no se bloquea.

10. Tools implementadas y función de cada una

Las tools (herramientas) son funciones que permiten a un LLM interactuar con sistemas externos, ejecutando acciones reales como consultar bases de datos o realizar cálculos complejos. El flujo usa dos tools principales: `query_match_metrics` y `retrieve_document_context`.

- **query_match_metrics:** consulta el estado agregado del partido a partir de los Parquet generados por Spark. Lee los eventos procesados y las métricas acumuladas, y devuelve un JSON con datos globales: total de eventos, tiros, goles, pases, porcentaje de acierto, duelos, equipo destacado, jugador destacado, equipo con mayor índice defensivo e intervalo de mayor intensidad. Dentro del workflow la usa el nodo metrics_agent. Este agente ya recibe las métricas de la ventana concreta, pero llama a esta tool para añadir contexto global del partido. Así el texto no se genera mirando solo cinco minutos aislados, sino comparando ese tramo con lo que está ocurriendo en el partido completo.
- **retrieve_document_context:** es la tool RAG. Su función es recuperar contexto documental desde data/docs/rag_corpus. Recibe una consulta textual construida por el metrics_agent, busca fragmentos relevantes en el corpus y devuelve texto con fuentes. Ese corpus contiene criterios para interpretar métricas futbolísticas: dominio con balón, presión, duelos, índice ofensivo, índice defensivo, intensidad y reglas de redacción. Esta tool la usa el rag_agent. Su salida se guarda como rag_context y después pasa al agente redactor. Gracias a esto, el LLM no redacta solo desde números, sino con apoyo táctico y explicativo.

El flujo empieza cuando Spark genera un snapshot y llama a generate_missing_report_segments. Esta función detecta qué ventanas de partido están cerradas y aún no tienen texto. Después entra el metrics_agent. Este agente toma las métricas de la ventana y llama a query_match_metrics para obtener la visión global del partido. Con ambas cosas construye la rag_query, una consulta enriquecida con datos del tramo y del partido completo.

Luego actúa el nodo rag_agent. Este agente recibe la rag_query y ejecuta retrieve_document_context. Recupera fragmentos del RAG documental y los deja en el estado como rag_context.

Por último, actúa el nodo writer_agent. Este agente construye el prompt final con métricas de ventana, eventos representativos, métricas acumuladas, métricas globales obtenidas con query_match_metrics y contexto RAG obtenido con retrieve_document_context. Con todo eso llama a Ollama para generar un fragmento narrativo en español. Si el LLM falla, se genera un texto fallback para no bloquear el pipeline.

El resultado final se guarda en output/reports/incremental_segments.parquet, incluyendo el texto, las métricas, el contexto RAG, el modelo usado y la traza del workflow. Luego el PDF y Streamlit reutilizan esos artefactos ya generados.

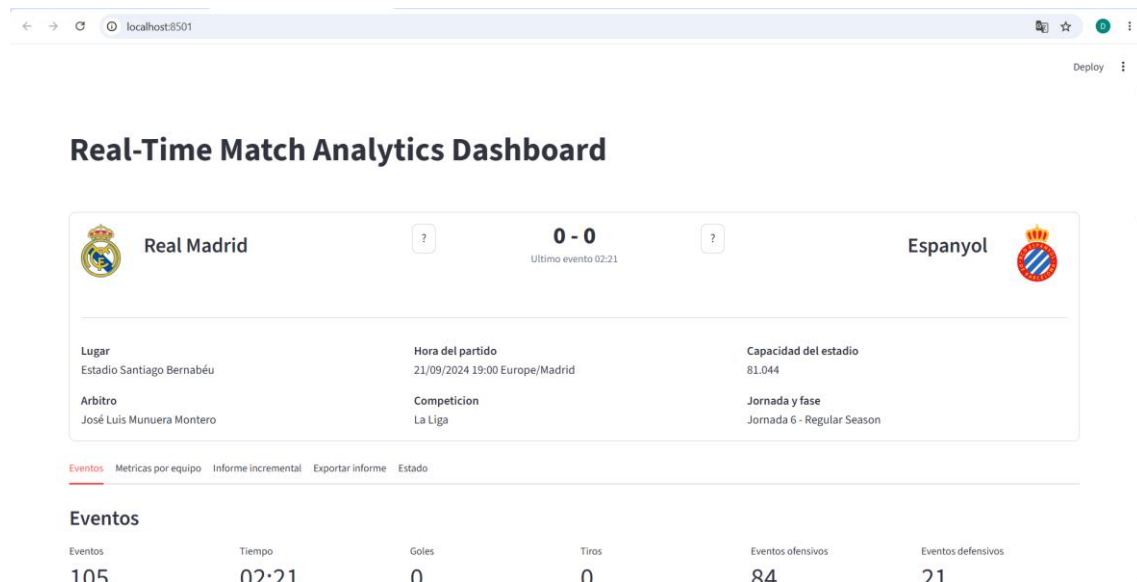
11. Visualización y Generación del Informe

La visualización y generación del informe se apoya en dos piezas principales: la aplicación Streamlit y el módulo de generación de archivos PDF. Ambas están separadas del procesamiento pesado. Es decir, Streamlit no consume Kafka directamente, no recalcula métricas con Spark y no llama al LLM. Su función es leer artefactos ya generados y presentarlos de forma clara.

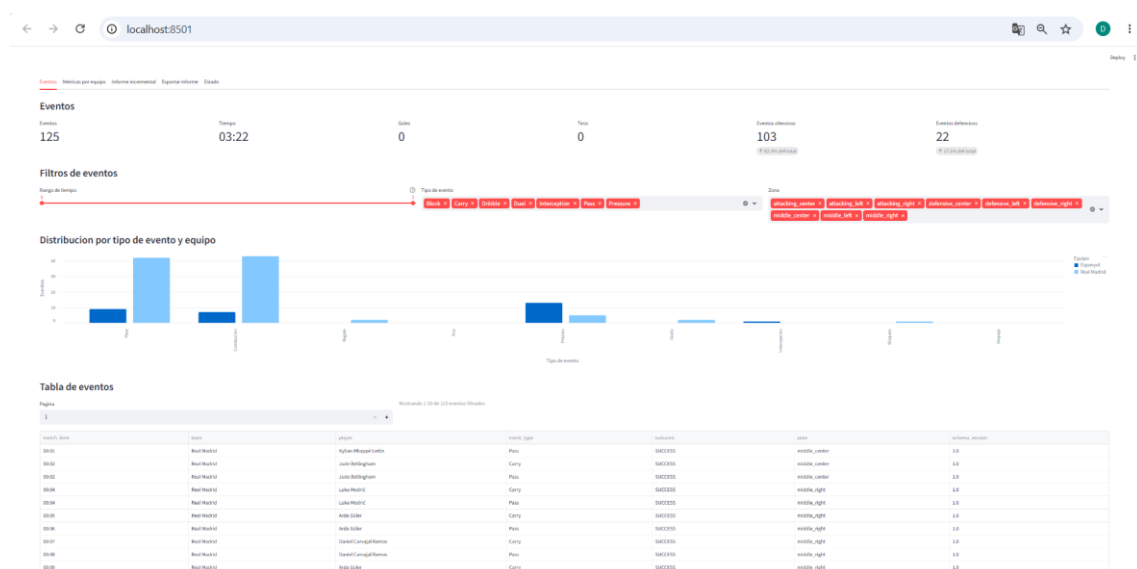
Usar Streamlit en este tipo de sistemas es beneficioso porque permite construir dashboards interactivos de forma rápida sobre datos procesados. En un proyecto de analítica en tiempo real, esto es especialmente útil ya que el usuario puede ver cómo evolucionan los eventos, las métricas y los informes sin tener que abrir ficheros Parquet ni revisar logs del pipeline. Además, Streamlit al ser un framework de Python encaja bien con PySpark, Pandas y PyArrow, que ya forman parte del stack del proyecto. Esto permite conectar fácilmente la capa de visualización con las salidas

del pipeline. La aplicación se encuentra en *src/app.py* y lee principalmente los archivos parquet que se generan tras el procesamiento de los datos de Spark Streaming.

A partir de esos ficheros, Streamlit muestra varias vistas. En cada una de ellas siempre muestra información complementaria a los eventos como información del partido (marcador, estadio, lugar, jornada...) y botones para visualizar información de cada equipo.



En la pestaña de eventos se ven los últimos eventos procesados, el minuto de partido, goles, tiros y distribución por tipo de evento.



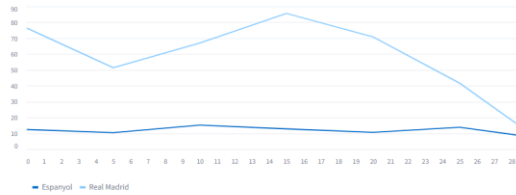
En la pestaña de métricas por equipo se comparan los equipos con métricas acumuladas, índice ofensivo, índice defensivo, porcentaje de pase, porcentaje de duelos y evolución por ventanas.

Detalle acumulado

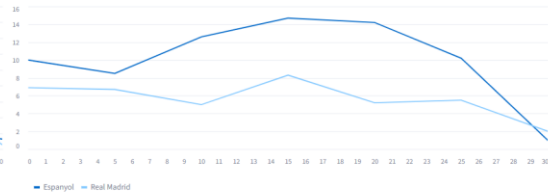
team	total_events	offensive_index	defensive_index	pass_success_pct	duel_success_pct	goals	passes	shots	pressures	duels	interceptions	blocks	clearances
Espanyol	279	81.6	71.2	61.45	21.43	0	83	0	92	14	5	10	7
Real Madrid	631	395.6	39.6	90.78	71.43	0	293	11	41	7	2	10	2

Indices ofensivo y defensivo

Indice ofensivo



Indice defensivo



En la pestaña de informe incremental se muestran los fragmentos generados por los agentes, junto con el modelo usado, la traza del workflow y el contexto RAG.

Minutos 0-5

Minutos 5-10

Ventana 5-10

En la ventana de 5 a 10 minutos, Real Madrid dominó el partido con una acumulación significativa de eventos ofensivos. El equipo acumuló 83 eventos en esta ventana, lo que refleja su control sobre el ritmo del juego y su capacidad para circular la pelota. Los pases fueron especialmente efectivos, con un porcentaje de éxito del 89.47%. Además, Luka Modrić tuvo una actuación destacada, con dos pasos exitosos en esta ventana. Sin embargo, el equipo no logró anotar goles en este periodo.

Acumulado del partido

En los primeros 10 minutos del partido, Real Madrid acumuló un total de 281 eventos, lo que refleja su dominio sobre el juego. El equipo acumuló 205 pases, con un porcentaje de éxito del 90.53%. Además, Luka Modrić tuvo una actuación destacada, con 31 participaciones en este periodo. Real Madrid también acumuló un total de 10 presiones y 4 duelos exitosos en esta ventana. En cuanto a la calidad de la posesión, el equipo demostró un alto porcentaje de éxito en los pases, lo que refleja su capacidad para circular la pelota y controlar el ritmo del juego.

Generado por: ollama/llama3.2 | Hora Madrid: 13/05/2026 17:42:20 Europe/Madrid

Contexto, fuentes y métricas

Minutos 10-15

Minutos 15-20

Minutos 20-25

Minutos 25-30

Por último, en la pestaña de exportación se permite descargar el PDF si ya ha habido textos de ventanas generados.

Eventos
Métricas por equipo
Informe incremental
Exportar informe
Estado

Informe evolutivo PDF

Informe PDF disponible.

Descargar PDF

La generación del informe no ocurre dentro de Streamlit, sino en el pipeline de streaming. Cuando Spark detecta una ventana cerrada, se ejecuta el workflow de agentes y se guarda un fragmento narrativo en `output/reports/incremental_segments.parquet`. Cada fragmento incluye el texto generado, las métricas usadas, el contexto RAG, el modelo LLM y la traza del proceso. Esto permite que el informe sea auditable: no solo se ve el resultado final, sino también qué datos y contexto han intervenido. Posteriormente, el módulo `src/report_generator.py` construye el PDF final. Este módulo no vuelve a llamar al LLM. Simplemente compone el informe a partir de los textos generados y las métricas calculadas ya existentes. El PDF incluye una lectura general del partido, resumen evolutivo, jugador destacado, tablas de métricas, gráficos de distribución y evolución, y los textos generados por ventana.

Informe automatico del partido

Generado a partir de metricas Spark y narrativas incrementales ya persistidas. Fecha: 2026-05-13 15:42:20

1. Metricas generales del partido

El partido acumula 295 eventos, 3 tiros y 0 goles. El mayor indice ofensivo corresponde a Real Madrid (135.4) y el mayor indice defensivo a Espanyol (19.5).

2. Lectura evolutiva

El informe evolutivo integra 2 ventanas narradas de 5 minutos de partido. El equipo con mayor indice ofensivo acumulado es Real Madrid (135.4), mientras que el mayor indice defensivo corresponde a Espanyol (19.5).

Equipo	Eventos	Ind. Of.	Ind. Def.	% Pase	% Duelo	Goles
Espanyol	82	25.0	19.5	65.38	25.0	0
Real Madrid	213	135.4	13.6	89.9	100.0	0

3. Jugador destacado

Jugador destacado: Éder Gabriel Militão (Real Madrid). Se destaca por su volumen e impacto: 31 intervenciones, 11 pases exitosos, 10 conducciones, 1 regates exitosos, 2 tiros, 0 goles, 3 presiones y 2 duelos ganados.

4. Textos por ventanas de 5 minutos

Bloques generados previamente a partir de ventanas cerradas de partido.

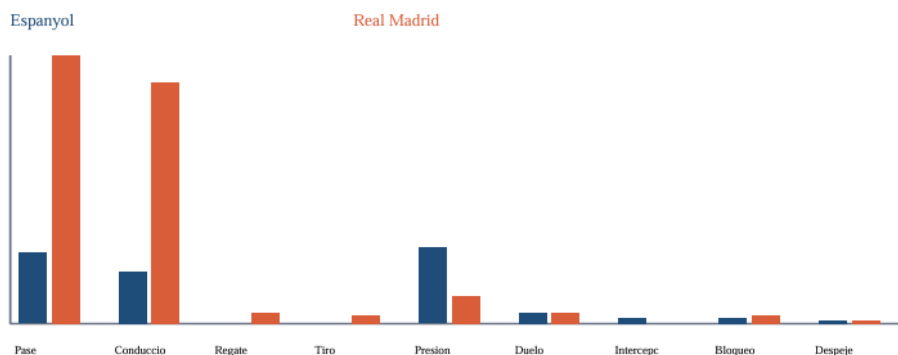
Minutos 0-5

Ventana 0-5

En la ventana inicial del partido, Real Madrid ha demostrado un dominio ofensivo significativo. Con 164 eventos de posesión en esta ventana, el equipo ha mantenido un alto porcentaje de acierto en pases (91.23%) y ha generado continuidad en sus ataques con conducciones y regates. Aunque no han anotado goles, la presencia de tiros y oportunidades de gol es evidente. En contraste, Espanyol ha sido más defensivo, con un indice de posesión inferior a

5. Distribucion de eventos por equipo

Distribucion de eventos por equipo



Desde la aplicación, Streamlit comprueba si ese fichero existe. Si está disponible, muestra un botón de descarga en la sección “Exportar Informe”. Por tanto, la app actúa como punto de acceso al informe final, pero no lo genera bajo demanda. Esto es una decisión importante porque evita que la interfaz bloquee al usuario con llamadas al LLM o procesos pesados. La generación queda en el backend de streaming, y la visualización queda en una capa ligera y reactiva.

En resumen, Streamlit aporta una interfaz rápida, clara e interactiva para monitorizar el sistema, revisar métricas en tiempo casi real y acceder al informe final. El PDF, por su parte, se genera de forma controlada desde los artefactos persistidos, garantizando que el contenido del informe está alineado con los datos procesados por Spark y con los textos producidos previamente por el workflow de agentes.

12. Decisiones de optimización

En este sistema hay varias decisiones de optimización importantes, buscando conseguir un pipeline estable, trazable y fácil de ejecutar de extremo a extremo. Las más importantes son:

- **Uso de snapshots temporizados en lugar de escribir en disco en cada microbatch.** Spark consume eventos continuamente desde Kafka, pero el sistema solo persiste resultados cada cierto tiempo (por defecto 30 segundos). Esto reduce operaciones de escritura, evita generar demasiados ficheros pequeños y hace que Streamlit lea datos más estables. En un sistema de streaming, escribir demasiado a menudo puede penalizar mucho el rendimiento, sobre todo cuando la interfaz está leyendo esos mismos artefactos.
- **Consolidar las salidas en un único fichero Parquet por tipo de artefacto.** Spark normalmente escribe directorios con múltiples part files, pero aquí se usa una función auxiliar que escribe temporalmente y después mueve un único .parquet final. Esto simplifica mucho la lectura desde pandas y Streamlit, porque la app no tiene que resolver directorios ni múltiples versiones. Es una decisión muy práctica para un proyecto académico y demostrable, aunque no sería la opción más escalable en producción.
- **Deduplicación por event_id.** El pipeline mantiene eventos en memoria indexados por identificador y después aplica dropDuplicates. Esto evita contar dos veces un evento si Spark reprocesa datos, si Kafka entrega mensajes ya leídos o si hay reinicios parciales. En métricas deportivas acumuladas, un duplicado puede distorsionar goles, tiros, pases o índices, así que esta decisión protege la calidad analítica.
- **Separar el cálculo pesado de la visualización.** Streamlit no consume Kafka, no ejecuta Spark y no llama al LLM. Solo lee Parquet y PDF ya generados. Esto hace que la interfaz sea más ligera, rápida y predecible. Si la app generase informes bajo demanda, cada usuario podría disparar llamadas al LLM o recalcular métricas, generando latencia y posibles inconsistencias.
- **Generar los textos narrativos en ventanas de 5 minutos.** Al agrupar los eventos en ventanas de partido, el sistema reduce el número de llamadas al LLM y genera textos con más contexto. Además, una ventana de 5 minutos permite resumir fases reales del juego: dominio, presión, transiciones, tiros o cambios de intensidad. Así, el informe no queda fragmentado en frases demasiado pequeñas, sino en bloques narrativos más útiles y legibles.
- **Uso de fallbacks.** Si LangGraph no está disponible, el proyecto ejecuta los mismos nodos con un runner local. Si Chroma o los embeddings no están disponibles, el RAG usa TF-IDF. Si Ollama falla, se genera un texto determinista. Estas decisiones hacen que el sistema sea robusto: una dependencia opcional no rompe todo el flujo.
- **Separar imágenes Docker por servicio.** Enfocado en la parte de despliegue, el productor solo necesita dependencias de Kafka, el streaming necesita Java, Spark, pandas, reportlab y GenAI, y el dashboard necesita Streamlit y librerías de lectura/visualización. Separar requisitos evita imágenes innecesariamente pesadas y reduce conflictos entre dependencias.

13. Limitaciones del sistema

Existen diferentes limitaciones en el sistema debido a requisitos técnicos más avanzados que no han podido ser desarrollados y al contexto del proyecto.

En primer lugar, una limitación importante del sistema es que la fuente de datos no es realmente en vivo. El proyecto simula un flujo en tiempo real a partir de un fichero estático de eventos. Kafka sí se usa como mecanismo real de ingesta, pero los eventos proceden de un dataset estático. Por tanto, el sistema reproduce el comportamiento de una arquitectura streaming, aunque no está conectado a un proveedor real de datos deportivos en directo. Si en futuro se quisiera conectar a un proveedor de datos real, sería una aproximación muy válida, pero habría que configurar y desarrollar algunas cosas más.

Por otro lado, hay una limitación en la estrategia de escritura. El sistema consolida cada salida en un único fichero Parquet para facilitar la lectura desde Streamlit. Esto simplifica mucho la demo, pero no es lo más escalable. Spark normalmente trabaja mejor con particiones y múltiples ficheros distribuidos. Para producción sería preferible particionar por partido, fecha, ventana o equipo, y usar una tabla tipo Delta Lake, Iceberg o Hudi.

El módulo GenAI también tiene limitaciones. El LLM depende de Ollama y de que el modelo configurado esté disponible localmente. Si Ollama no está levantado o el modelo no responde correctamente, el sistema genera un fallback determinista. Esto mantiene el pipeline funcionando, pero la calidad narrativa del fallback es menor que la de un texto generado por el modelo. El RAG es documental y local. Recupera contexto desde Markdown internos, no desde una base de conocimiento amplia ni actualizada automáticamente. Esto es positivo para la trazabilidad, pero limita la variedad del contexto. Además, si el corpus no cubre una situación táctica concreta, el agente redactor tendrá menos apoyo interpretativo.

Por último, el sistema está orientado a un único partido. La arquitectura podría extenderse a varios partidos, pero habría que adaptar particionado, almacenamiento, identificación de topics o claves Kafka, dashboards multi-partido y generación de informes por entidad. Actualmente está optimizado para explicar bien un flujo completo, no para operar una competición completa en paralelo.

14. Conclusiones

El proyecto demuestra una solución completa de analítica deportiva en tiempo real, integrando ingesta, procesamiento, visualización y generación automática de informes. A partir de un fichero estático de eventos StatsBomb, el sistema simula un flujo en tiempo real mediante Kafka, procesa los datos con PySpark Structured Streaming y transforma los eventos en métricas interpretables para el análisis de un partido.

Una de las principales fortalezas de la solución es la separación clara de responsabilidades. Kafka actúa como capa de ingesta, Spark se encarga del procesamiento y cálculo de métricas, Parquet funciona como capa de persistencia intermedia, Streamlit permite visualizar los resultados y el módulo GenAI genera fragmentos narrativos apoyados en agentes, tools y RAG. Esta división hace que el sistema sea más mantenible, trazable y fácil de explicar.

El uso de métricas como el índice ofensivo, índice defensivo, porcentaje de pase, duelos ganados, tiros, goles y presiones permite construir una lectura deportiva más rica que un simple conteo de

eventos. Además, el enfoque por ventanas de 5 minutos aporta una visión evolutiva del partido, permitiendo detectar fases de dominio, resistencia, presión o transición.

La parte GenAI aporta valor porque no genera texto de forma aislada. El workflow de agentes organiza el proceso: primero se preparan métricas, después se consulta contexto RAG y finalmente se redacta el fragmento con el LLM. Las tools conectan el modelo con datos reales del pipeline y con documentos de interpretación futbolística, reduciendo el riesgo de respuestas genéricas o inventadas.

Streamlit resulta especialmente útil como capa final, ya que permite consultar eventos, métricas, evolución por ventanas, fragmentos generados y el PDF final desde una interfaz sencilla. Además, al no ejecutar procesos pesados ni llamadas al LLM, mantiene una separación adecuada entre backend de procesamiento y frontend de visualización.

En conjunto, el proyecto cumple el objetivo de construir un pipeline extremo a extremo con tecnologías actuales de Big Data y GenAI. Aunque existen limitaciones, como el uso de una fuente simulada, métricas heurísticas y una arquitectura pensada para un único partido, la solución es coherente, funcional y extensible. Como evolución futura, podría conectarse a una fuente real de datos en directo, incorporar modelos analíticos más avanzados y ampliar el sistema para gestionar múltiples partidos o competiciones.

15. Consideraciones finales

Todo el código fuente se encuentra en el repositorio:

<https://github.com/DaniLopez23/real-time-match-simulator>

En el repositorio existe un README.md y otros archivos Markdown que explican toda la infraestructura e instrucciones de despliegue mediante Docker.

El siguiente enlace es una video demostración de la App Streamlit con todas sus funcionalidades:

<https://youtu.be/beBMt9hh9E4>